

Quiero que me ayudes a construir una herramienta de análisis estático basada en reglas Semgrep. Trabaja en fases y pídemela confirmación antes de pasar a la siguiente.

REPOSITORIO DE DESTINO

- El repositorio del proyecto es: <https://github.com/pablojimz/Repo-reglas-SEMGREP-y-YARA.git>
- TODA la estructura de reglas (carpetas de reglas por lenguaje, registry, licencias, etc.) debe crearse dentro de una carpeta llamada SEMGREP en la raíz de ese repo, no en la raíz directamente. Es decir: /SEMGREP/rules/..., /SEMGREP/registry.json, /SEMGREP/NOTICE.md, etc.

CONTEXTO

- Voy a importar reglas desde <https://github.com/trailofbits/semgrep-rules> (licencia AGPLv3)
- El repo es privado y permite descarga de solo lectura a terceros, pero solo yo/personas autorizadas podemos añadir o quitar reglas
- Cada PR en repos consumidores debe descargar solo las reglas del lenguaje afectado y lanzar el análisis

FASE 1 — Clonar y auditar reglas

1. Clona <https://github.com/trailofbits/semgrep-rules> en una carpeta temporal
2. Para cada regla (.yaml + su .test.yaml asociado), analízala contra este checklist y genera un informe en markdown (rules-audit-report.md) con una fila por regla:
 - id de la regla y lenguaje
 - ¿el patrón es demasiado amplio (alto riesgo de falsos positivos) o demasiado estrecho (poco útil)?
 - ¿tiene test cases y realmente pasan con `semgrep --test?`
 - ¿el severity/confidence declarado es coherente con el patrón?
 - ¿hay algo sospechoso: excepciones/exclusiones silenciosas, patrones que ignoran casos obvios, referencias rotas o poco fiables en metadata.references?
 - veredicto: APROBAR / REVISAR MANUALMENTE / RECHAZAR, con una línea justificando por qué
3. No modifiques ni descartes ninguna regla todavía, solo genera el informe
4. Muéstrame un resumen (cuántas aprobadas, cuántas a revisar, cuántas rechazadas) antes de continuar

FASE 2 — Estructurar el repo (dentro de la carpeta SEMGREP)

1. Dentro de /SEMGREP en <https://github.com/pablojimz/Repo-reglas-SEMGREP-y-YARA.git>, crea la estructura:
 - /SEMGREP/rules/third-party/trailofbits// (reglas importadas, con su LICENSE AGPLv3 dentro de esa carpeta)
 - /SEMGREP/rules/custom// (reglas propias, licencia a definir por mí)
 - /SEMGREP/registry.json (índice: id, lenguaje, ruta, origen, versión/hash upstream, fecha de última revisión)
 - /SEMGREP/CODEOWNERS
 - /SEMGREP/CONTRIBUTING.md

2. Copia solo las reglas con veredicto APROBAR de la Fase 1 a `rules/third-party/trailofbits/`, respetando su estructura por lenguaje
3. Deja un `/SEMGREP/NOTICE.md` explicando qué partes son AGPLv3 (third-party/) y qué partes no
4. Propón (no implementes todavía) 3-5 reglas nuevas específicas de seguridad que no existan ya en el conjunto importado, con su justificación

FASE 3 — Workflows de GitHub Actions

1. Workflow `"sync-upstream.yml"`: programado semanalmente, hace fetch de cambios en `trailofbits/semgrep-rules`, calcula el diff de reglas nuevas/modificadas, repite el checklist de la Fase 1 sobre las reglas nuevas, y abre un PR automático a main con las aprobadas (las dudosas quedan marcadas para revisión manual). Todo dentro de la ruta `/SEMGREP`
2. Workflow `"pr-analysis.yml"` (para repos consumidores, o como reusable workflow): al abrir/actualizar un PR, detecta los lenguajes tocados en el diff, descarga desde <https://github.com/pablojimz/Repo-reglas-SEMGREP-y-YARA.git> (carpeta SEMGREP, usando un token/deploy-key de solo lectura) solo las reglas de esos lenguajes, ejecuta `semgrep --config <ruleset> --baseline-commit <base>` sobre el diff, y publica los resultados como comentario en el PR (bloqueando el merge si hay findings de severidad alta/crítica)
3. Configura branch protection en main: PR obligatorio + review aprobado, sin push directo

Empieza por la Fase 1 y espera mi confirmación antes de seguir.